

ES Futures

Machine Learning Price Direction Predictor

Author: Rahul

Built with Python, yfinance, pandas, scikit-learn

1. Executive Summary

This project applies machine learning to predict the next-day price direction of S&P 500 E-mini Futures (ES=F). Using one year of daily OHLCV data, we engineer four financial features — previous return, 5-day moving average, 20-day moving average, and 5-day volatility — and train a Random Forest classifier. The model achieves 53.19% accuracy, beating the 50% random baseline, which is a meaningful starting point for a quantitative trading strategy.

Metric	Value
Asset	ES=F (S&P 500 E-mini Futures)
Data period	1 year of daily data
Total trading days	234
Model	Random Forest Classifier (100 trees)
Train / Test split	80% / 20%
Model accuracy	53.19%
Random baseline	50.00%
Up days	128
Down days	106

2. Data & Methodology

2.1 Data Source

Data was sourced from Yahoo Finance via the yfinance Python library. We downloaded one year of daily OHLCV (Open, High, Low, Close, Volume) data for the ES=F ticker, which represents the S&P; 500 E-mini Futures contract — the most liquid derivative instrument in the world with daily notional volume exceeding \$200 billion.

2.2 Feature Engineering

Four features were engineered from the raw price data to serve as inputs to the machine learning model:

Feature	Formula	What It Captures
prev_return	$\text{Close}[t] / \text{Close}[t-1] - 1$	Short-term momentum (yesterday's return)
mavg_5	Rolling 5-day mean of Close	Weekly price trend
mavg_20	Rolling 20-day mean of Close	Monthly price trend
volatility	Rolling 5-day std of daily return	Market uncertainty / risk regime

2.3 Target Variable

The target is a binary variable: 1 if the next day's closing price is higher than today's (UP), and 0 if it is lower (DOWN). This is a classification problem — the model outputs a probability and predicts either UP or DOWN.

2.4 Train / Test Split

Data was split 80/20 with shuffle=False to preserve temporal order. This is critical for financial data — shuffling would leak future information into the training set, producing falsely optimistic results. The model trains on the first ~187 days and is tested on the most recent ~47 days it has never seen.

3. Statistical Analysis

Before building the model, we analyze the statistical properties of ES returns. Understanding the distribution of returns is essential for building robust models and managing risk.

Statistic	Value	Interpretation
Mean daily return	0.1037%	~25% annualized — strong bull market
Median daily return	0.1161%	Slightly above mean — a few large down days pulled mean lower
Std deviation	0.7593%	Typical daily move is under 0.76%

Skewness	-0.1163	Slight negative — market falls faster than it rises
Kurtosis	1.5090	Fat tails — extreme moves more frequent than expected
Best single day	+2.8568%	Captured a strong rally day
Worst single day	-2.7142%	Captured a sharp selloff

3.1 Key Statistical Insights

Fat Tails (Kurtosis = 1.51): A normal distribution has kurtosis of 0. ES futures show kurtosis of 1.51, meaning extreme moves (crashes and rallies) occur more often than classical models predict. This is why risk models that assume normality consistently underestimate tail risk.

Negative Skew (-0.12): The slight negative skew confirms the well-known asymmetry of equity markets — prices rise slowly and fall sharply. Any trading strategy must account for this asymmetry in position sizing.

Volatility Clustering: The rolling 20-day volatility chart reveals clear clustering — calm periods (vol ~0.4%) followed by turbulent periods (vol ~1.25%). This phenomenon, known as GARCH effects, means volatility today predicts volatility tomorrow. Future model iterations should incorporate this.

4. Machine Learning Model Results

4.1 Model Performance

Metric	DOWN (0)	UP (1)	Overall
Precision	44%	65%	57% (weighted)
Recall	63%	46%	53%
F1-Score	52%	54%	53%
Support	19 days	28 days	47 days
Accuracy			53.19%

4.2 Feature Importance

The Random Forest model revealed approximately equal importance across all four features, suggesting no single indicator dominates. This is a healthy result — it indicates the model is not overfit to one signal.

Feature	Importance	Interpretation
prev_return	25.55%	Yesterday's return is equally predictive as longer-term signals
mavg_5	23.87%	Short-term trend carries moderate signal
mavg_20	25.54%	Monthly trend equally important as daily momentum
volatility	25.04%	Volatility regime helps calibrate predictions

4.3 Interpreting 53% Accuracy

In financial ML, 53% accuracy is not a failure — it is a starting point. The market is one of the hardest prediction problems in existence. Any strategy above 50% can be made profitable with proper position sizing and risk management. Professional quant funds typically target 52-56% accuracy at the individual signal level, combining many such signals into a portfolio.

5. Core Code

5.1 Data Loading & Feature Engineering

```
import yfinance as yf
import pandas as pd

df = yf.download('ES=F', period='1y', interval='1d')
df['daily_return'] = df['Close'].pct_change()
df['prev_return'] = df['daily_return'].shift(1)
df['mavg_5'] = df['Close'].rolling(5).mean()
df['mavg_20'] = df['Close'].rolling(20).mean()
df['volatility'] = df['daily_return'].rolling(5).std()
df['target'] = (df['daily_return'].shift(-1) > 0).astype(int)
df = df.dropna()
```

5.2 Model Training

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

features = ['prev_return', 'mavg_5', 'mavg_20', 'volatility']
X, y = df[features], df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print('Accuracy:', accuracy_score(y_test, predictions))
```

6. Planned Improvements

The current model uses four simple features. The following improvements are planned to push accuracy toward 55-57%:

Priority	Improvement	Expected Gain
High	Add RSI and MACD indicators	+1-2% accuracy

High	Add order book imbalance features	+1-2% accuracy
Medium	Use 3-5 years of data instead of 1	+1% accuracy
Medium	Tune Random Forest hyperparameters	+0.5-1% accuracy
Medium	Try XGBoost or LightGBM	+1-2% accuracy
Low	Implement LSTM neural network	+1-3% accuracy
Low	Add options market data (VIX, put/call ratio)	+1-2% accuracy

7. References

1. Fischer, T. & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*.
2. Cont, R., Kukanov, A. & Stoikov, S. (2014). The price impact of order book events. *Journal of Financial Econometrics*.
3. Jegadeesh, N. & Titman, S. (1993). Returns to buying winners and selling losers: Implications for stock market efficiency. *Journal of Finance*.
4. Fama, E. (1970). Efficient capital markets: A review of theory and empirical work. *Journal of Finance*.
5. yfinance Python library: <https://github.com/ranaroussi/yfinance>
6. scikit-learn documentation: <https://scikit-learn.org>

This project was built as a learning exercise in quantitative finance and machine learning. Nothing in this report constitutes financial advice.